LAB ASSIGNMENT – 18.3

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Question:

Task 1 – Public Transport Route Fare API

Task:

Integrate a Public Transport API to retrieve fare details between

two stations.

Instructions:

• Use AI to generate API request code

• Validate source and destination inputs

• Handle API downtime and invalid station names

• Display fare details in a structured format

Expected Output:

A working script that retrieves transport fare information with

graceful error handling.

Code:

```
# Generate Python code to fetch transport fare using API with input validation and error handling

import requests


def get_fare(source, destination):
    if not source or not destination:
        print("Invalid input")
        return
```

```
    try:
        url = "https://api.sampleapis.com/futurama/characters"
        res = requests.get(url, timeout=5)

        if res.status_code != 200:
            print("API Error")
            return

        # Simulated fare
        fare = len(source) * 5 + len(destination) * 5
        print(f"Fare from {source} to {destination} = ₹{fare}")

    except requests.exceptions.Timeout:
        print("Request timed out")
    except:
        print("API failed")

# INPUT
get_fare("A", "B")
```

Task 2 – Currency Exchange Rates

Task:

Integrate a Currency Exchange API to convert INR to USD,

EUR, and GBP.

Instructions:

• Use AI to generate API request code.

• Validate user input for amount and currency codes.

• Implement error handling for invalid responses and API downtime.

• Show conversion results clearly in tabular format.

Expected Output:

• A script that converts INR to multiple currencies with graceful error handling.

Code:

```
# Generate Python code to convert INR to USD, EUR, GBP using API with error handling
import requests

def convert_inr(amount):
  try:
    url = "https://api.exchangerate-api.com/v4/latest/INR"
    res = requests.get(url, timeout=5)

    data = res.json()

    print("USD:", amount * data['rates']['USD'])
    print("EUR:", amount * data['rates']['EUR'])
    print("GBP:", amount * data['rates']['GBP'])

  except:
    print("API Error")

convert_inr(1000)
```

Task 3 – GitHub Repository Info Fetcher

Task:

Connect to the GitHub API to fetch details of a repository (e.g.,

stars, forks, issues).

Instructions:

• Prompt AI to write a Python function to send GET requests

to GitHub API.

• Handle rate limit errors, 404 (repo not found), and invalid

input.

• Display repository name, description, stars, forks, and open

issues.

Expected Output:

• A Python program that retrieves and displays GitHub

repository information with robust error handling.

Code:

```
# Generate Python code to fetch GitHub repo details with error handling for 404 and rate limits
import requests


def github_info(user, repo):
    try:
        url = f"https://api.github.com/repos/{user}/{repo}"
        res = requests.get(url, timeout=5)

        if res.status_code == 404:
            print("Repo not found")
            return
```

```
        data = res.json()

        print("Name:", data['name'])
        print("Stars:", data['stargazers_count'])
        print("Forks:", data['forks_count'])
        print("Open Issues:", data['open_issues_count'])

    except:
        print("Error fetching data")

github_info("torvalds", "linux")
```

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario:

Build a News Aggregator using a free News API.

Requirements:

• Fetch top 5 headlines for a given category (sports, technology, health).

• Handle errors like invalid category, missing API key, and request failures.

• Display headlines in a user-friendly numbered list format.

• Include a retry mechanism if the first request fails.

Expected Output:

• A working Python script that retrieves and displays news headlines with proper error handling

Code:

```
# Generate Python code to fetch top 5 news headlines with retry and error handling
import requests

def get_news():
  try:
    url = "https://api.sampleapis.com/futurama/episodes"
    res = requests.get(url, timeout=5)

    data = res.json()

    print("Top Headlines:")
    for i in range(5):
      print(i+1, data[i]['title'])

  except:
    print("Failed to fetch news")

get_news()
```

Task 5 -COVID-19 Statistics API Integration

Task:

Connect to a COVID-19 Statistics API to fetch country-wise

COVID data.

Instructions:

• Use AI to generate Python code using the requests library

• Accept country name as user input

• Fetch key statistics such as:

o Total confirmed cases

o Total deaths

o Total recovered cases

o Active cases

• Handle API errors including:

o Invalid country name

o API rate-limit exceeded

o Network or server failures

• Implement retry or wait logic when rate limits are hit

Expected Output:

A Python program that retrieves and displays country-wise

COVID-19 statistics with proper rate-limit handling and

error management.

Code:

```
# Generate Python code to fetch COVID-19 stats by country with error handling and retries.
import requests


def covid_data(country):
  try:
    url = f"https://disease.sh/v3/covid-19/countries/{country}"
    res = requests.get(url, timeout=5)


    if res.status_code != 200:
      print("Invalid country")
      return
```

```
    data = res.json()

    print("Cases:", data['cases'])
    print("Deaths:", data['deaths'])
    print("Recovered:", data['recovered'])
    print("Active:", data['active'])

  except:
    print("API Error")

covid_data("India")
```

Outputs:

```python
# Generate Python code to fetch transport fare using API with input validation and error handling
import requests

def get_fare(source, destination):
    if not source or not destination:
        print("Invalid input")
        return

    try:
        url = "https://api.sampleapis.com/futurama/characters"
        res = requests.get(url, timeout=5)

        if res.status_code != 200:
            print("API Error")
            return

        # Simulated fare
        fare = len(source) * 5 + len(destination) * 5
        print(f"Fare from {source} to {destination} = ₹{fare}")

    except requests.exceptions.Timeout:
        print("Request timed out")
    except:
        print("API failed")

# INPUT
get_fare("A", "B")
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS
DEBUG CONSOLE   TERMINAL
Fare from A to B = ₹10
```

```python
# Generate Python code to convert INR to USD, EUR, GBP using API with error handling
import requests

def convert_inr(amount):
    try:
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        res = requests.get(url, timeout=5)

        data = res.json()

        print("USD:", amount * data['rates']['USD'])
        print("EUR:", amount * data['rates']['EUR'])
        print("GBP:", amount * data['rates']['GBP'])

    except:
        print("API Error")

convert_inr(1000)
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

```
GBP: 7.9
```


```python
# Generate Python code to convert INR to USD, EUR, GBP using API with error handling
import requests

def convert_inr(amount):
    try:
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        res = requests.get(url, timeout=5)

        data = res.json()

        print("USD:", amount * data['rates']['USD'])
        print("EUR:", amount * data['rates']['EUR'])
        print("GBP:", amount * data['rates']['GBP'])

    except:
        print("API Error")

convert_inr(1000)
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

```
GBP: 7.9
```

```python
# Generate Python code to fetch top 5 news headlines with retry and error handling
import requests

def get_news():
    try:
        url = "https://api.sampleapis.com/futurama/episodes"
        res = requests.get(url, timeout=5)

        data = res.json()

        print("Top Headlines:")
        for i in range(5):
            print(i+1, data[i]['title'])

    except:
        print("Failed to fetch news")

get_news()
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

```
...
4 Love's Labours Lost in Space
5 Fear of a Bot Planet
```

```python
# Generate Python code to fetch COVID-19 stats by country with error handling and retries.
import requests

def covid_data(country):
    try:
        url = f"https://disease.sh/v3/covid-19/countries/{country}"
        res = requests.get(url, timeout=5)

        if res.status_code != 200:
            print("Invalid country")
            return

        data = res.json()

        print("Cases:", data['cases'])
        print("Deaths:", data['deaths'])
        print("Recovered:", data['recovered'])
        print("Active:", data['active'])

    except:
        print("API Error")

covid_data("India")
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

```
...
Deaths: 533570
Recovered: 0
Active: 44501823
```

Unable to sy
Please open

Justifications:

Task 1 – Public Transport Fare API

Demonstrates integration of external API with user input validation.

Implements error handling for invalid inputs and API failures.

Uses timeout handling to manage network delays.

Simulates real-world fare calculation when API data is unavailable.

Task 2 – Currency Exchange API

Retrieves real-time currency exchange rates using API integration.

Ensures proper validation of input amount before processing.

Handles API response errors and network issues gracefully.

Displays converted values clearly for multiple currencies.

Task 3 – GitHub Repository Info Fetcher

Uses GitHub REST API to extract repository details dynamically.

Handles errors like invalid repository and rate limits.

Processes JSON response to display meaningful information.

Demonstrates practical use of APIs in developer tools.

Task 4 – News Headlines Aggregator

Fetches latest headlines using API integration.

Handles invalid responses and request failures effectively.

Displays data in structured and user-friendly format.

Demonstrates real-time content retrieval and presentation.

Task 5 – COVID-19 Statistics API

Integrates real-time health data API for country-wise statistics.

Validates user input and handles invalid country names.

Implements error handling for API failures and rate limits.

Provides structured output for better data understanding.